Accelerating Mamba Kernel on Trainium: Performance Improvements and Hardware Limits

Batu El, *Institute for Computational and Mathematical Engineering, Stanford University*

Overview

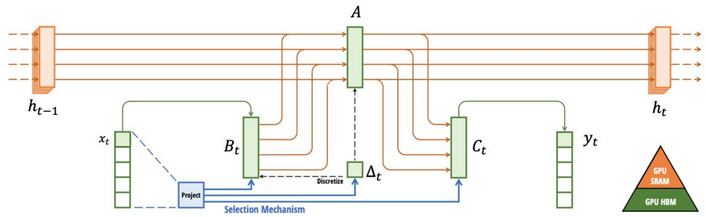
Linear-time sequence modeling. Mamba is a selective state space model (S6) that processes sequences in linear time, offering an efficient alternative to attention-based Transformers whose cost grows quadratically with sequence length.

Mamba Kernel optimization for Trainium 2. We adapt and optimize the Mamba selective scan kernel for NeuronCore-v3 through loop reordering, sequence-length tiling, and SBUF-aware tuning.

Linear scaling. We analyze whether the optimized kernel retains Mamba's linear scaling property across sequence lengths.

Hardware bottleneck analysis. We identify fundamental limits in the Trainium architecture and analyze what architectural changes would be needed to unlock further acceleration.

Selective State Space Model



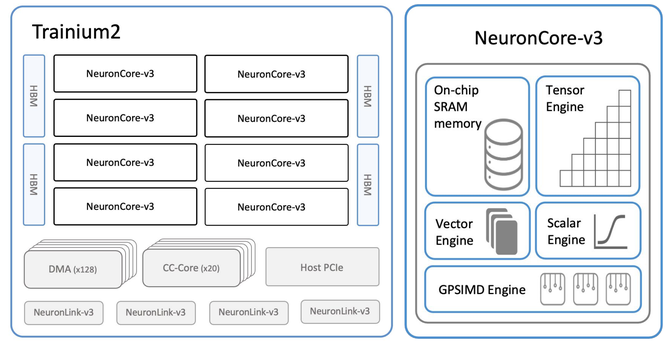
Input-dependent parameterization. The input x is projected through a selection mechanism to produce step-specific parameters B, C, and delta, making the state dynamics adapt to input content rather than remaining fixed.

Parallelization Opportunities. The recurrence is independent across batch elements (B) and channels (D). Computation is parallelizable (with a reduction to form the output) over the state dimension (N), and is causally ordered along the sequence length (L).

Hardware-aware state expansion. The full materialized state is large, but kernel design expands it only within fast on-chip memory (SRAM) rather than spilling to slower HBM.

Ref: https://arxiv.org/pdf/2312.00752

Trainium2 Architecture

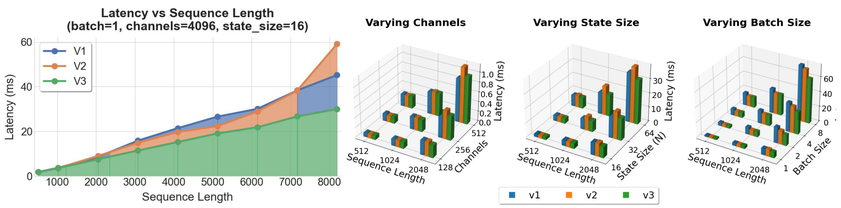


Trainium2 chip and NeuronCore-v3 architecture. The Trainium2 chip contains eight NeuronCore-v3 compute cores connected to HBM via 128 DMA engines, 20 collective-compute cores, and NeuronLink-v3 interconnects. Each NeuronCore-v3 includes on-chip SRAM, a Tensor Engine for matrix operations, Vector and Scalar Engines for element-wise computation, and a general-purpose SIMD Engine.

Ref: https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/guides/architecture/trainium2_arch.html

Results & Discussion

Baselines

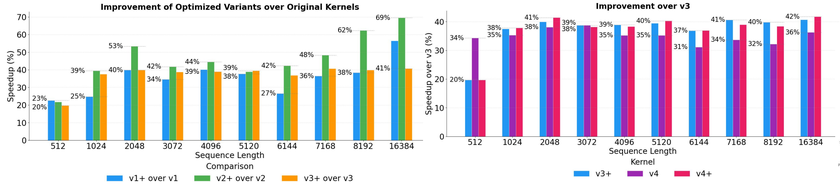


V1 (Baseline). Loop order *[batch, state, channel]*. Loads delta and u for every state iteration. 3D accumulator indexed by channel tile.

V2 (Loop Reordering). Loop order *[batch, channel, state]. Loads d*elta and u outside the state loop so each tile is loaded once and reused across all N state iterations. Reduces HBM memory traffic by shrinking the accumulator from 3D to 2D.

V3 (Sequence Tiling). Loop order *[batch, channel, state, seq\tile].* Tiles the sequence dimension into 512-element chunks on top of V2's reordering, reducing on-chip SBUF pressure to avoid spilling to HBM. Scan state is propagated across tiles via an explicit carry variable (scan_init).

Performance Improvements



V1+ (V1 + BF16). Loop order *[batch, state, channel]*. Adds BF16 to trigger VectorE 2x performance mode for nisa.tensor_tensor operations.

V2+ (V2 + BF16 + Hoisted deltaU). Loop order *[batch, channel, state]*. Adds BF16. Hoists the state-invariant product *deltaU = delta * u* outside the state loop, eliminating (N - 1) redundant *VectorE* multiplies per channel tile.

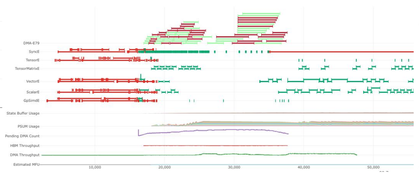
V3+ (V3 + BF16 + Larger Tile). Loop order *[batch, channel, state, seq\tile]*. Adds BF16 and an larger sequence tile size (1024); BF16's halved element size combined with Trainium2's larger SBUF enables the bigger tile.

V4 (BF16 + Loop Reorder). Loop order *[batch, channel, seq\tile, state]*. Per-*seq\tile* output accumulation/store. Loads delta/u per-tile. which shrinks SBUF but introduces DMA overhead at long sequences. Adds BF16.

V4+ (v4 + DMA Fix + Larger Tile). Loads full-sequence delta/u loading eliminate DMA overhead. Increases *seq\tile* size from 512 to 1024.

Hardware Limits

V4+ Neuron Profile for seq_len=2048



Version	v1	v1+	v2	v2+	v3	v3+	v4	v4+
Total (ms)	8.603	5.183	9.012	**4.211**	7.559	4.546	4.671	4.435
Tensor %	2.7	1.2	79.2	48.8	**94.6**	45.9	1.3	47.2
Vector %	79.1	88.1	66	95.6	82.6	92.9	**97.8**	95
Scalar %	14.4	21.9	35	73.1	21.5	72.1	35.4	**73.6**
DMA %	60.8	63.1	6.8	7.9	9.2	6.9	18.4	**7.1**
Sync %	2.1	0.5	0.9	0.3	2.3	0.4	**6.2**	0.4
Active %	98.8	99.2	99.3	99.5	99.6	99.4	99.7	**99.9**
HBM Read (MB)	1437.1	1091	72.5	67.6	72.8	**67.6**	156.2	**67.6**
HBM Write (MB)	379.6	49.3	**33.6**	**33.6**	**33.6**	**33.6**	52.3	**33.6**
MBU %	29.49	**30.73**	1.64	3.36	1.97	3.11	6.24	3.19
HW FLOPS (M)	33.6	16.8	**1073.7**	536.9	**1073.7**	536.9	16.8	536.9
Arith Intensity	0.02	0.01	**10.13**	5.31	10.09	5.31	0.08	5.31

99.9%+ active utilization leaves no idle cycles to exploit. Every cycle is already occupied by useful work across the vector, scalar, or tensor engines. **v3+ and v4+ converge to almost identical profiles despite different algorithmic approaches.** Two different optimization paths hit the same hardware wall.

Larger SBUF capacity would enable larger tiles and fewer scan iterations. Doubling SBUF would halving the number of tensor_tensor_scan invocation (which consumes ~71% of active time in v3+/v4+). **A faster scalar engine or hardware-assisted scan would overcome the bottleneck.** The scalar engine is the dominant pipeline stage. A wider or higher-clocked scalar unit, or a dedicated hardware prefix-scan instruction that handles multi-step carry internally, would directly reduce the time spent on the serial dependency chain.